COMPUTER SCIENCE & DA

Data Structure through Python

Linked List

Discussion Notes

DPP- 01

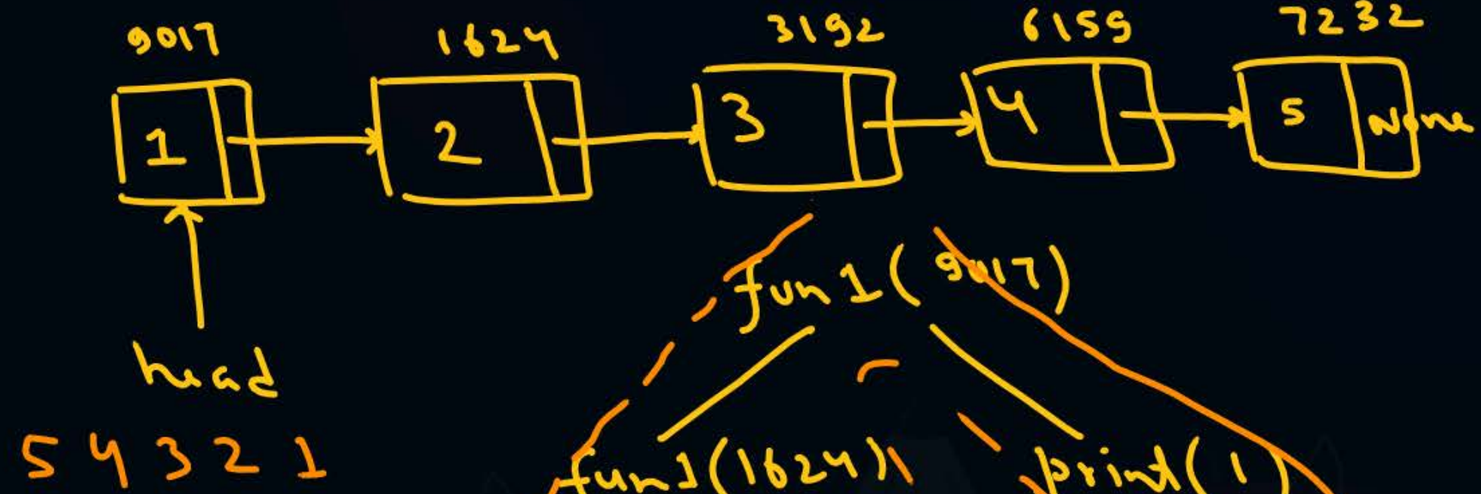


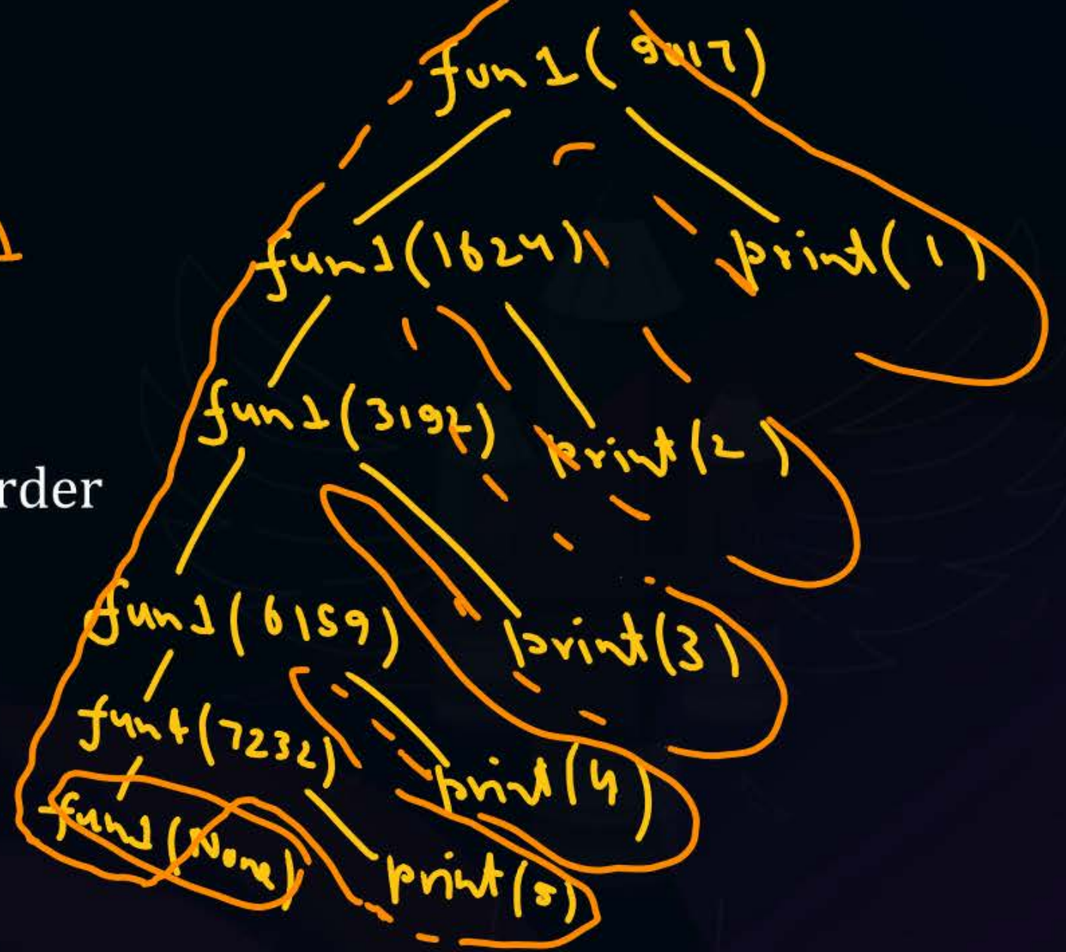
Mohd Kashif Khan sir

[MCQ]

#Q What does the following function do for a given Linked List with first node as head ?

```
def fun1(head):
    if head is None:
        return
fun1(head.next)
print(head.data, end=" ")
```





A Prints all nodes of linked lists

B Prints all nodes of linked list in reverse order

C Prints alternate nodes of Linked List

D Prints alternate nodes in reverse order

[MCQ]

#Q The following function takes a single-linked list of integers as a parameter and rearranges the elements of the list. The function is called with the list containing the integers 1, 2, 3, 4, 5, 6, 7 in the given order. What will be the contents of the list after the function completes execution?

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None
def rearrange(head):
    if head is None or head.next is None:
        return
    p = head
    q = head.next
        while q is not None:
        # Swap values of p and q
        temp = p.value
        p.value = q.value
```



2 1 4 3 6 5 7

```
            q.value = temp
        # Move p and q to next pair of nodes
            p = q.next
            if p is not None:
                q = p.next
            else:
                q = None
    # Helper function to print the linked list
        def print_list(head):
        current = head
    while current is not None:
        print(current.value, end=" ")
        current = current.next
    print()
    # Example usage:
    if __name__ == "__main__":
```

```
# Creating a linked list: 1 → 2 → 3 → 4 → 5
head = Node(1)
head.next = Node(2)
head.next.next = Node(3)
head.next.next.next = Node(4)
head.next.next.next.next = Node(5)
print("Original list:")
print_list(head)
rearrange(head)
print("Rearranged list:")
print_list(head)
```

A 1,2,3,4,5,6,7

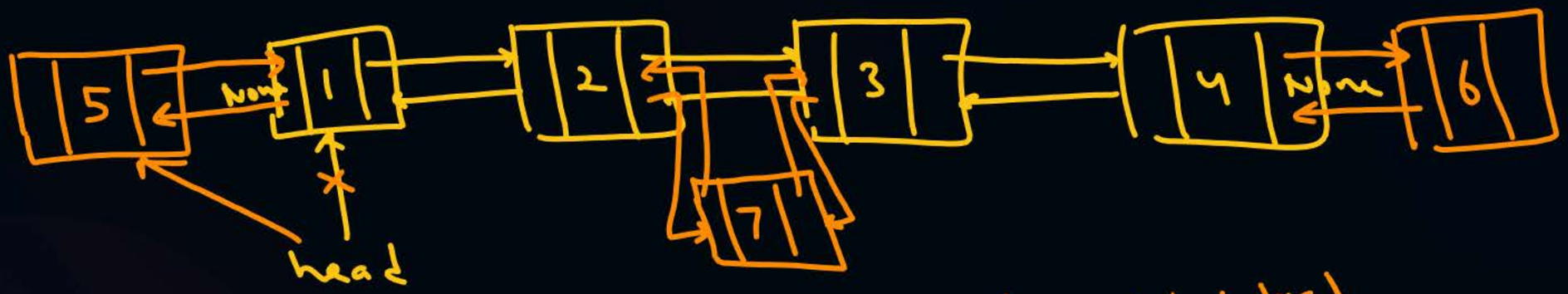
B 2,1,4,3,6,5,7

C 1,3,2,5,4,7,6

D 2,3,4,5,6,7,1

[MCQ]

#Q In a doubly linked list, the number of pointers affected for an insertion operation will be



Insert in beginning : 2 + 1 (head pointer)
= 3

Insert in the end = 2

Insert in the middle = 4

A 0

B 5

C 1

D None of these

[MCQ]

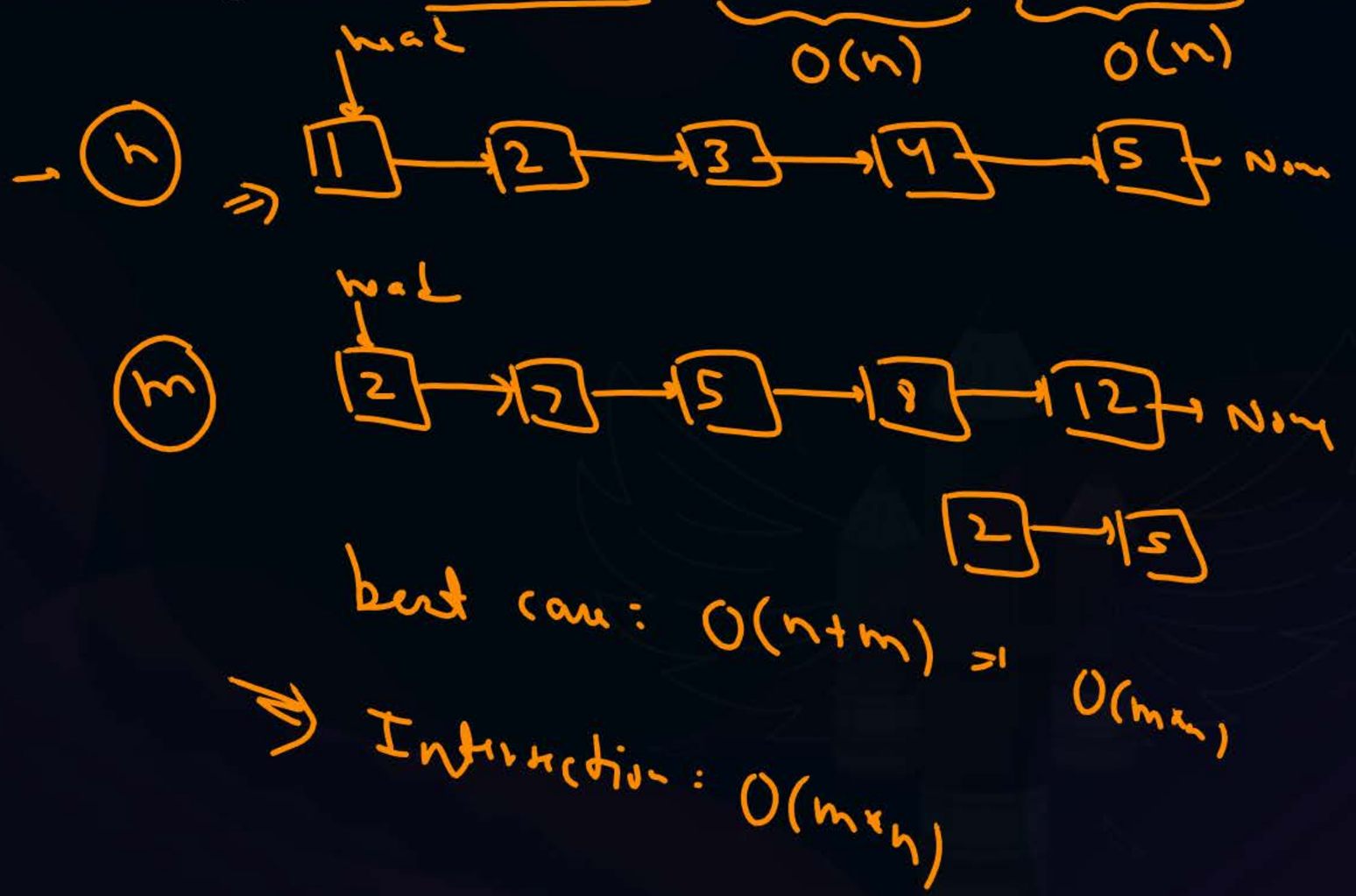
#Q Suppose each set is represented as a linked list with elements in arbitrary order. Which of the operations among union, intersection, membership, cardinality will be the slowest ?

A union

B membership

C cardinality

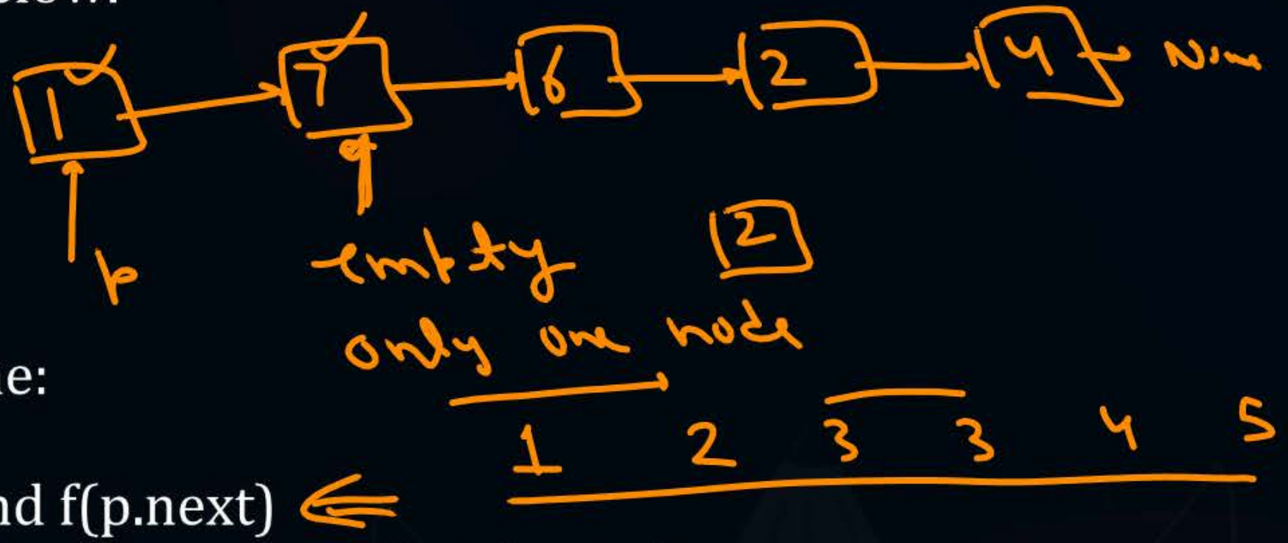
D union, intersection



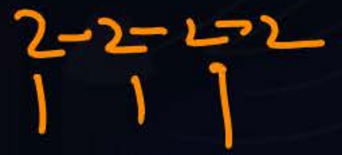
[MCQ]

#Q Consider the function f defined below.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
    def f(p):
        if p is None or p.next is None:
            return True
        return p.data <= p.next.data and f(p.next)
```



For a given linked list p, the function f returns 1 if and only if



A Not all elements in the list have the same data value

B The elements in the list are sorted in nondecreasing order of data value

C The elements in the list are sorted in nonincreasing order of data value

D None of them

[MCQ]

#Q Which of the following points is/are true about Linked List data structure when it is compared with array ?



A Arrays have better cache locality that can make them better in terms of performance.



B It is easy to insert and delete elements in Linked List

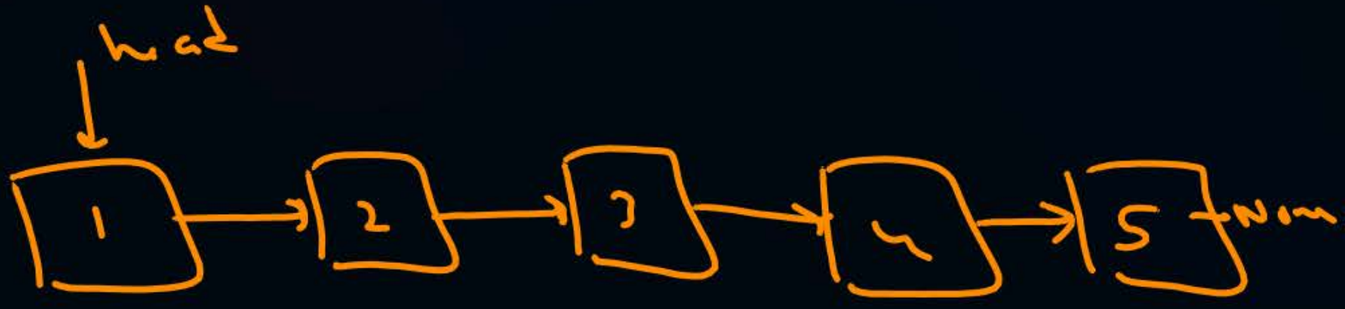
C Random access is not allowed in a typical implementation of Linked Lists

D The size of array has to be pre-decided, linked lists can change their size any time.

E All of the above

[MCQ]

#Q In the worst case, the number of comparisons needed to search a singly linked list of length n for a given element is



A log n

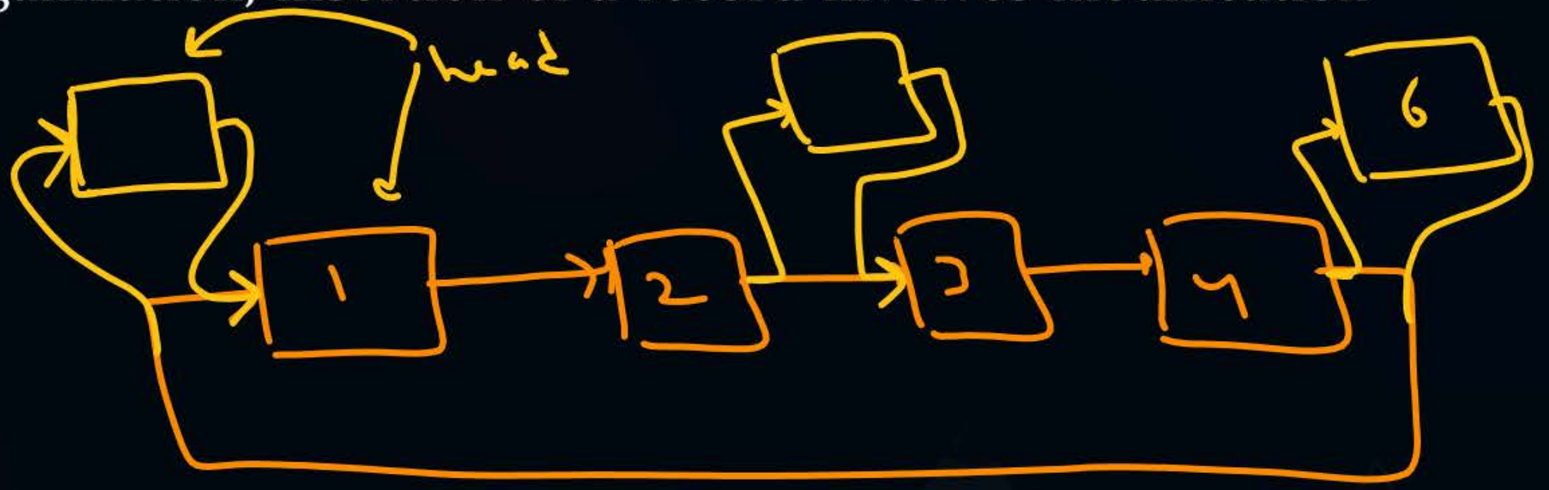
B n/2

C log n – 1

D n

[MCQ]

#Q In a circular linked list organization, insertion of a record involves modification of



Insert in beg : 2 + 1(head)

Insert in end : 2

Insert in middle : 2

A One pointer

B Two pointers

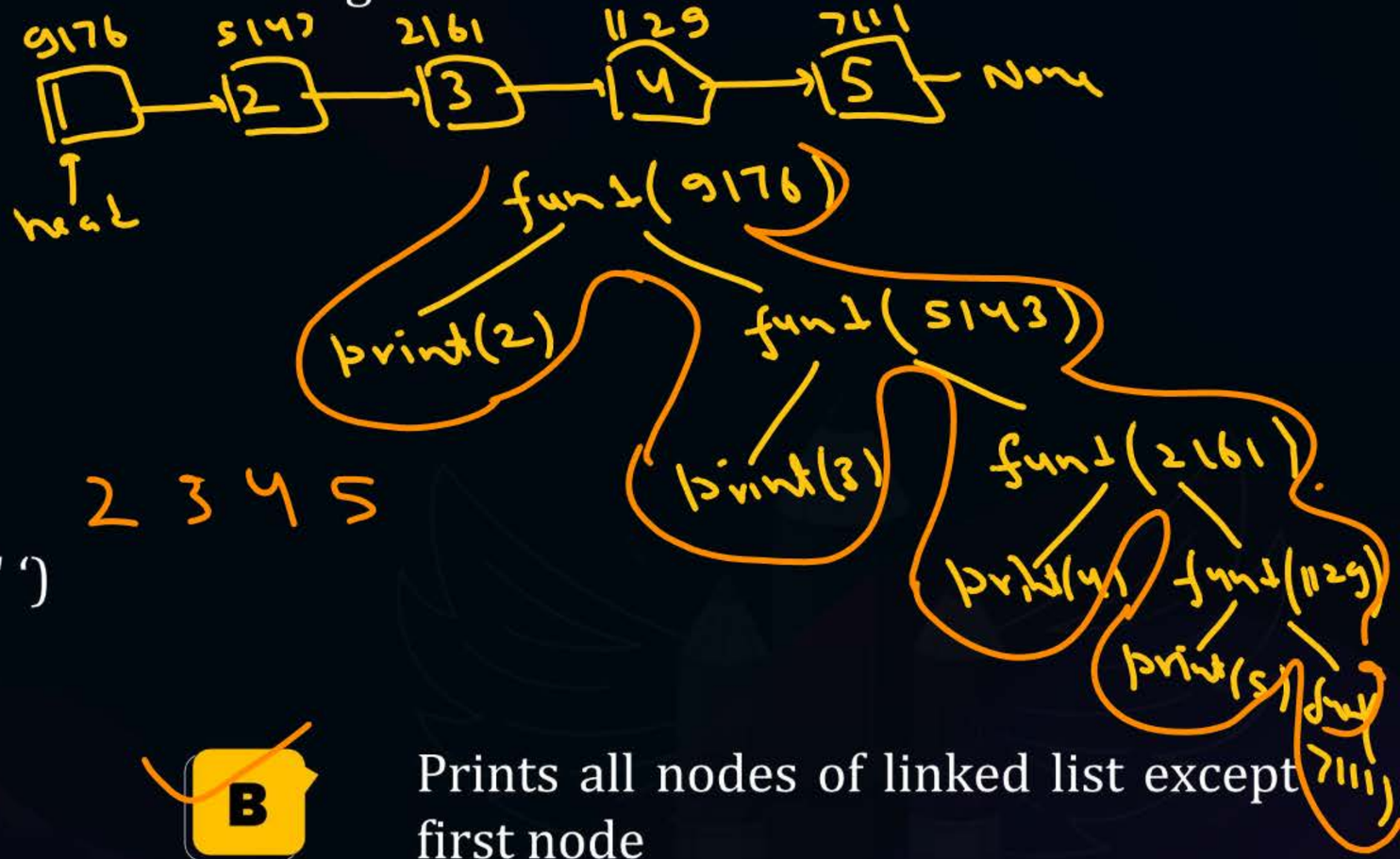
C Multiple pointers

D No pointer

[MCQ]

#Q What does the following function do for a given Linked List with first node as head ?

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
    def fun1(head):
        if head.next is None:
            return
        print(head.next.data, end=' ')
        fun1(head.next)
```



A Prints all nodes of linked lists

B Prints all nodes of linked list except first node

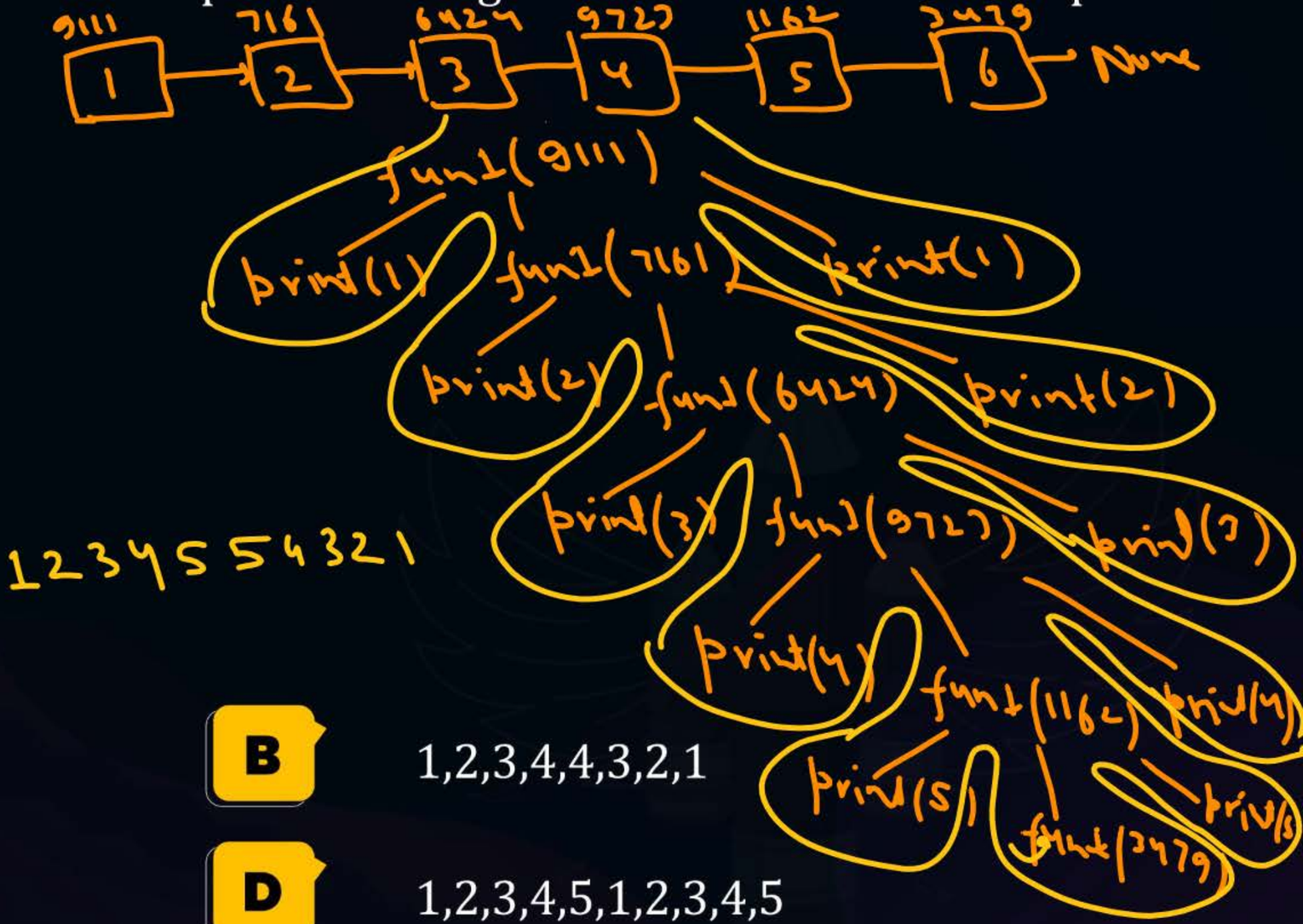
C Prints alternate nodes of Linked List

D Prints all nodes of linked list except last node

[MCQ]

#Q What does the following function print for a given Linked List with input 1,2,3,4,5,6 ?

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
def fun1(head):
    if head.next is None:
        return
    print(head.data, end=' ')
    fun1(head.next)
    print(head.data, end=' ')
```



A 1,2,3,4,5,6,6,5,4,3,2,1

B 1,2,3,4,4,3,2,1

C 1,2,3,4,5,5,4,3,2,1

D 1,2,3,4,5,1,2,3,4,5

THANK - YOU